

Binary Trees — Pattern Cheatsheet

Python · LeetCode patterns · companion to the Sliding Window and Linked List sheets

0. The Node, and the Only Two Questions You Ever Ask

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Almost every tree problem is recursion, and almost every tree recursion is answered by two questions:

- 1. **What do I need to send DOWN to my children?** (context from ancestors: current depth, running sum, allowed value range, path so far) → this becomes a *function parameter*.
- 2. **What do I need to send UP to my parent?** (a summary of my whole subtree: height, sum, count, "is it valid") → this becomes the *return value*.

The single most useful reframe: when you write `dfs(node)`, do not trace the recursion. Instead *assume the recursive calls already work correctly* on `node.left` and `node.right`, and only write the logic that combines their two answers into yours. This is the whole skill. Trees are easier than linked lists once you stop tracing.

The universal skeleton:

```
def dfs(node, stuff_from_above):
    if not node:
        return identity_value # 1. base case: the empty tree
    L = dfs(node.left, ...) # 2. trust the recursion
    R = dfs(node.right, ...)
    return combine(node.val, L, R) # 3. combine
```

Base case discipline: your base case is `node is None`, *not* "node is a leaf". Handling `None` means leaves get handled for free (both children return the identity value). Only special-case a leaf when the problem is genuinely about leaves — e.g. root-to-*leaf* path sums. Leaf test is `not node.left and not node.right`.

1. Traversal Orders — What "Pre/In/Post" Actually Means

All three visit every node once, $O(n)$. The only difference is *when you do your work relative to the recursive calls*.

```
def preorder(node):
    # work BEFORE children -> top-down
    if not node: return
    visit(node)
    preorder(node.left); preorder(node.right)

def inorder(node):
    # work BETWEEN children -> sorted order in a BST
    if not node: return
    inorder(node.left); visit(node); inorder(node.right)

def postorder(node):
    # work AFTER children -> bottom-up
    if not node: return
    postorder(node.left); postorder(node.right); visit(node)
```

Order	Use it when	Typical problems
Preorder	Info flows <i>down</i> ; the parent must be processed before the child can be judged	Path sum, path collection, serialize, "good nodes" (max on path so far), depth tracking, copy a tree
Inorder	The tree is a BST	Validate BST, kth smallest, BST → sorted array, min absolute difference, mode in BST
Postorder	Info flows <i>up</i> ; you need both subtree answers before you can decide	Height, diameter, balanced, LCA, max path sum, subtree sums, prune tree, count univalued subtrees

Interview one-liner worth memorizing: "Inorder traversal of a BST yields values in sorted ascending order." A huge fraction of BST problems are just "run inorder and do the obvious array thing, but streaming."

2. Postorder / Bottom-Up — The Workhorse Pattern

2a. Max depth (the template every bottom-up problem mutates)

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Height of empty tree = 0, so height of a leaf = 1. Time $O(n)$, space $O(h)$.

2b. Balanced tree — the "sentinel return" trick LC 110

Naive approach calls `height()` at every node $\rightarrow O(n^2)$. Instead, compute height and validity in *one* pass by overloading the return value with a sentinel:

```
def isBalanced(root):
    def height(node):
        if not node:
            return 0
        L = height(node.left)
        if L == -1: return -1          # early bail: subtree already unbalanced
        R = height(node.right)
        if R == -1: return -1
        if abs(L - R) > 1: return -1  # I am the culprit
        return 1 + max(L, R)
    return height(root) != -1
```

Pattern name: "return the answer, track the violation." When a problem asks a yes/no question that depends on a numeric quantity, return the number and reserve one impossible value (`-1`) to mean "already failed". Turns $O(n^2)$ into $O(n)$.

2c. Diameter — the "global + return" split LC 543

This is the pattern people get wrong most often, because **what you report upward is not the answer**.

```
def diameterOfBinaryTree(root):
    best = 0
    def height(node):
        nonlocal best
        if not node:
            return 0
        L = height(node.left)
        R = height(node.right)
        best = max(best, L + R)      # ANSWER: path bends at this node (in edges)
        return 1 + max(L, R)        # UPWARD: a path must go straight through me
    height(root)
    return best
```

Why two different values? A path that *turns* at node X (goes down-left then down-right) can never be extended by X's parent — you'd have to visit X twice. So the bent path is only ever a candidate for the global answer; the value you hand your parent must be a straight downward path. Any problem with the phrase "path may or may not pass through the root" is this pattern.

2d. Max path sum — same shape, plus clamping LC 124

```
def maxPathSum(root):
    best = float('-inf')
    def gain(node):
        nonlocal best
        if not node:
            return 0
        L = max(gain(node.left), 0)    # clamp: a negative subtree is worth SKIPPING
        R = max(gain(node.right), 0)
        best = max(best, node.val + L + R)
        return node.val + max(L, R)
    gain(root)
    return best
```

Two differences from diameter: the `max(..., 0)` clamp (you may decline a subtree, since you're free to end the path at `node`), and `best` starts at `-inf` because all values can be negative.

3. Preorder / Top-Down — Carrying State Downward

3a. Root-to-leaf path sum LC 112

```
def hasPathSum(root, target):
    if not root:
        return False # empty tree has no root-to-leaf path
    if not root.left and not root.right: # leaf: this is the decision point
        return target == root.val
    rem = target - root.val
    return hasPathSum(root.left, rem) or hasPathSum(root.right, rem)
```

Why the leaf case is mandatory here: if you only checked `if not root`, a node with one child would "succeed" down the `None` side and let you stop mid-path. The problem says root-to-*leaf*, so a leaf must be explicitly recognised.

3b. Collecting all paths — DFS + backtracking LC 113 · 257

```
def pathSum(root, target):
    res, path = [], []
    def dfs(node, rem):
        if not node:
            return
        path.append(node.val) # choose
        rem -= node.val
        if not node.left and not node.right and rem == 0:
            res.append(path[:]) # SNAPSHOT (copy!), not path
        else:
            dfs(node.left, rem)
            dfs(node.right, rem)
        path.pop() # un-choose / backtrack
    dfs(root, target)
    return res
```

Two classic bugs in five lines. (1) `res.append(path)` appends a *reference* — every entry in `res` ends up empty after backtracking. Always `path[:]` or `list(path)`. (2) Forgetting `path.pop()`, or putting it inside an `if`, so it doesn't run on every exit route. Rule: one `append` at the top, exactly one `pop` at the bottom, unconditionally.

3c. Passing a constraint down — "count good nodes" LC 1448

```
def goodNodes(root):
    def dfs(node, mx):
        if not node:
            return 0
        good = 1 if node.val >= mx else 0
        mx = max(mx, node.val) # local rebind: each branch gets its own
        return good + dfs(node.left, mx) + dfs(node.right, mx)
    return dfs(root, float('-inf'))
```

Note that passing state as a *parameter* gives you backtracking for free — reassigning `mx` inside one call cannot leak into a sibling call. That's why parameters are preferred over mutable shared state whenever the state is a scalar.

4. BFS / Level Order — When "Level" Appears in the Problem

```
from collections import deque

def levelOrder(root):
    if not root:
        return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)): # <-- snapshot the size BEFORE appending
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

The one line that matters: `for _ in range(len(q))`. Python evaluates `len(q)` once, before the loop body starts appending children, which is exactly what separates one level from the next. If you wrote `while q:` without the inner loop you'd get a flat traversal with no level boundaries.

Problem	Change to the template
Right side view 199	Append <code>level[-1]</code> only
Level averages 637	Append <code>sum(level)/len(level)</code>
Zigzag order 103	<code>res.append(level if depth % 2 == 0 else level[::-1])</code>
Max width / largest value per level 515	<code>max(level)</code>
Min depth 111	Return current depth at the <i>first</i> leaf you dequeue — BFS beats DFS here, it stops early
Connect next pointers 116/117	Link <code>prev.next = node</code> within the inner loop

5. Binary Search Trees

BST invariant: for every node, **all** values in the left subtree $<$ `node.val` $<$ **all** values in the right subtree. Note "all", not "the immediate child".

5a. Validate BST — bounds, not parent comparison [LC 98](#)

```
def isValidBST(root):
    def ok(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return ok(node.left, low, node.val) and ok(node.right, node.val, high)
    return ok(root, float('-inf'), float('inf'))
```

The trap: checking only `node.left.val < node.val` passes this invalid tree — 6 is in the left subtree of 10 but greater than it. Every node must know the full open interval it's allowed to live in, narrowed as you descend.

10 / \ 5 15 \ 6 <- violates the BST property w.r.t. 10, not w.r.t. its parent 5

5b. Kth smallest — iterative inorder with early exit [LC 230](#)

```
def kthSmallest(root, k):
    stack, curr = [], root
    while stack or curr:
        while curr:
            # dive left, remembering the way back
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        # leftmost unvisited = next smallest
        k -= 1
        if k == 0:
            return curr.val
        curr = curr.right
        # done with this node; go right once
    return -1
```

Worth knowing as the canonical **iterative inorder**. It's $O(h + k)$, not $O(n)$ — you stop as soon as you've counted k . Recursive inorder with a counter also works, but you need `nonlocal` and a way to short-circuit.

5c. Search, insert, and LCA in a BST are all $O(h)$ loops

```
def searchBST(root, target):
    while root and root.val != target:
        root = root.left if target < root.val else root.right
    return root

def lcaBST(root, p, q):
    # LC 235
    while root:
        if p.val < root.val and q.val < root.val:
            root = root.left
            # both in left subtree
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            return root
    # split point (or one equals root)
```

6. LCA in a General Binary Tree [LC 236](#)

```
def lowestCommonAncestor(root, p, q):
    if not root or root is p or root is q:
        return root
    L = lowestCommonAncestor(root.left, p, q)
    R = lowestCommonAncestor(root.right, p, q)
    if L and R:
        return root          # p and q found on opposite sides -> I am the LCA
    return L or R           # both on one side (or neither) -> pass it upward
```

Read the return value carefully — it's overloaded: "either the LCA if I found it, or whichever of p/q exists in my subtree, or None." That overloading is what makes six lines sufficient. It's the same "return the answer, track the violation" idea from the balanced-tree problem: a single return slot doing double duty.

7. Structural Problems — Compare, Invert, Construct, Serialize

7a. Two-tree recursion (same tree / symmetric / subtree) 100 · 101 · 572

```
def isSameTree(a, b):
    if not a and not b: return True
    if not a or not b: return False      # exactly one is None -> shape differs
    return a.val == b.val and isSameTree(a.left, b.left) \
           and isSameTree(a.right, b.right)

def isSymmetric(root):
    # mirror: compare left-vs-right crosswise
    def mirror(a, b):
        if not a and not b: return True
        if not a or not b: return False
        return a.val == b.val and mirror(a.left, b.right) and mirror(a.right, b.left)
    return mirror(root.left, root.right) if root else True

def isSubtree(root, sub):
    # try isSameTree at every node -> O(n*m)
    if not root: return False
    return isSameTree(root, sub) or isSubtree(root.left, sub) or isSubtree(root.right, sub)
```

7b. Invert a tree LC 226

```
def invertTree(root):
    if not root:
        return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

7c. Construct from preorder + inorder LC 105

```
def buildTree(preorder, inorder):
    idx = {v: i for i, v in enumerate(inorder)} # value -> position, O(1) lookup
    it = iter(preorder)
    def build(lo, hi):
        if lo > hi:
            return None
        val = next(it)          # preorder gives roots in exactly the order we need
        node = TreeNode(val)
        m = idx[val]            # split inorder at the root
        node.left = build(lo, m - 1)
        node.right = build(m + 1, hi)
        return node
    return build(0, len(inorder) - 1)
```

Preorder's first element is always the root of the current segment; inorder tells you how many nodes fall left vs. right of it. Building left before right is what keeps the shared `it` iterator in sync — $O(n)$ total, versus $O(n^2)$ if you slice arrays instead.

7d. Serialize / deserialize with a null sentinel LC 297

```
def serialize(root):
    out = []
    def dfs(node):
        if not node:
            out.append('#'); return      # explicit nulls make the shape unambiguous
            out.append(str(node.val))
            dfs(node.left); dfs(node.right)
    dfs(root)
    return ','.join(out)

def deserialize(data):
    it = iter(data.split(','))
    def build():
        val = next(it)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()
```

Why the # matters: preorder alone does not determine a tree — [1, 2] could be 2 as a left child or a right child. Recording nulls removes the ambiguity, which is why one traversal suffices here but two are needed in LC 105.

8. Iterative DFS — and When You're Forced To Use It

```
def preorderIterative(root):
    if not root: return []
    res, stack = [], [root]
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.right: stack.append(node.right)    # push RIGHT first...
        if node.left:  stack.append(node.left)     # ...so LEFT pops first
    return res
```

Postorder iteratively: run the above with `left` pushed before `right`, then reverse the result — that gives you (root, right, left) reversed = (left, right, root). Inorder iteratively is the dive-left loop in §5b.

Python-specific gotcha: the default recursion limit is 1000. A skewed tree with 10^5 nodes has height 10^5 and will blow the stack. If constraints allow a degenerate (linked-list-shaped) tree, either go iterative or call `sys.setrecursionlimit(...)`. Interviewers occasionally ask for the iterative version specifically to see if you know why.

9. Complexity Reference

Operation	Time	Space
Any full DFS traversal	$O(n)$	$O(h)$ call stack — $O(\log n)$ balanced, $O(n)$ skewed
BFS level order	$O(n)$	$O(w)$, max width; up to $O(n/2) = O(n)$ at the bottom level
BST search / insert / LCA	$O(h)$	$O(1)$ iterative
Kth smallest in BST	$O(h + k)$	$O(h)$
Build from pre + in (with hashmap)	$O(n)$	$O(n)$
Subtree of another tree (naive)	$O(n \cdot m)$	$O(h)$
Serialize / deserialize	$O(n)$	$O(n)$

For a balanced tree $h = \Theta(\log n)$; for a degenerate tree $h = n$. Always state which one you're assuming — " $O(\log n)$ if balanced, $O(n)$ worst case" is the answer interviewers want.

10. Bug Table — Check These Before You Submit

Symptom	Cause / fix
AttributeError: 'NoneType' has no attribute 'val'	Missing <code>if not node: return ...</code> base case, or you recursed without a null guard
All collected paths come back empty	<code>res.append(path)</code> stores a reference. Use <code>path[:]</code>
Paths contain leftovers from other branches	Missing or conditional <code>path.pop()</code> . One append at top, one pop at bottom, always
<code>UnboundLocalError</code> on your global <code>best</code>	Missing <code>nonlocal best</code> inside the inner function (or use a one-element list / <code>self.best</code>)
Invalid BST accepted	You compared to the parent instead of carrying <code>(low, high)</code> bounds down
Off-by-one on diameter	Diameter is counted in edges : <code>L + R</code> , not <code>L + R + 1</code> . If the problem counts nodes, add 1
Max path sum too small on negative trees	Missing <code>max(child, 0)</code> clamp, or <code>best</code> initialised to 0 instead of <code>-inf</code>
BFS levels all merged into one list	<code>len(q)</code> not snapshotted before the inner loop
Root-to-leaf answer wrong on one-child nodes	You treated <code>node is None</code> as the leaf case. Leaf is <code>not node.left</code> and <code>not node.right</code>
TLE on "is balanced" / "is subtree"	Recomputing height inside the recursion $\rightarrow O(n^2)$. Fold the check into one pass
<code>RecursionError</code> on large input	Skewed tree deeper than 1000. Go iterative or raise the limit
LCA returns the wrong node with duplicate values	Compare identity (<code>node is p</code>) not value, when the problem hands you node references

11. Problem → Pattern Map

If the problem says...	Reach for
"depth", "height", "balanced"	Postorder, return height, sentinel <code>-1</code> for violations
"path may not pass through root", "diameter", "max path sum"	Global variable + different upward return (\$2c, \$2d)
"root-to-leaf", "all paths", "path sum equals"	Preorder DFS, pass remaining target down, backtrack with append/pop
"level", "row", "zigzag", "right side view", "shallowest"	BFS with <code>len(q)</code> snapshot
"binary search tree", "kth smallest", "sorted"	Inorder traversal, or $O(h)$ bounded descent
"lowest common ancestor"	BST $\rightarrow$ descend by comparison; general tree $\rightarrow$ overloaded postorder return
"same", "symmetric", "subtree of"	Two-pointer style recursion on a <i>pair</i> of nodes
"construct from traversals", "serialize"	Preorder for roots + inorder for the split, or preorder with <code>#</code> sentinels
"count subtrees where...", "prune", "sum of subtree"	Postorder returning a tuple / struct of aggregates
"O(1) space traversal"	Morris traversal (threading) — rare, but know it exists

12. Pre-Submit Checklist

1. **Is `root = None` handled?** Almost every LeetCode tree problem allows an empty tree.
2. **Which direction does information flow?** Down $\rightarrow$ parameter, preorder. Up $\rightarrow$ return value, postorder. Both $\rightarrow$ you likely need a global plus a return.
3. **What's my identity value?** 0 for height/sum, `True` for conjunctions, `-inf` for maxima, `None` for node-returning functions.
4. **Am I recomputing anything?** If the same subtree's height/sum is computed more than once, fold it into a single postorder pass.
5. **Edge cases:** single node; all values negative; nodes with exactly one child; duplicate values; completely skewed tree; `p` is an ancestor of `q` (LCA problems).
6. **Did I state complexity in terms of h and n?** "O(n) time, O(h) space, which is O(log n) balanced and O(n) worst case."